

Binary Tree Zigzag Level Order Traversal

Difficulty: Medium

Patterns: Tree, Binary Tree, Breadth-First Search, Queue, Deque

LeetCode: <https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/description/>

1. Problem Summary

Given the root of a binary tree, return the node values level by level. The first level is read from left to right, the next level from right to left, and the direction keeps alternating.

For example:

- Input: `root = [3,9,20,null,null,15,7]`
- Output: `[[3], [20,9], [15,7]]`

An empty tree should return an empty list.

2. Constraints and Observations

- Constraint: The number of nodes is in the range `[0, 2000]`. What it implies: A linear traversal is easily fast enough, and recursion depth is avoidable with an iterative BFS.
- Constraint: `-100 <= Node.val <= 100`. What it implies: Values do not affect the algorithm; we only care about tree structure and output ordering.
- Observation: The answer is grouped by depth, so this is still a level-order traversal problem.
- Observation: Zigzag order changes how we store each level's values, not how we discover the next level's nodes.

3. Pattern Recognition

The core pattern is Breadth-First Search on a binary tree.

Normal level-order traversal already gives exactly the grouping we need: root level first, then its children, then grandchildren, and so on. The only extra state is a boolean direction flag:

- `left_to_right = True` for the first level.
- Flip the flag after every level.
- When the flag is false, place values for that level in reverse order.

4. Naive Approach

A first idea is:

1. Run a normal traversal to collect every node with its depth.
2. Group values by depth.
3. Reverse every odd-depth group at the end.

This is correct, but it separates the work into extra bookkeeping. It still takes $O(n)$ time, but it stores depth metadata or grouped lists before applying the zigzag rule. In an interview, we can do the same idea more directly while performing BFS.

5. Key Intuition

The queue should always process nodes in normal left-to-right BFS order because that keeps child discovery simple and correct.

The zigzag behavior belongs only to the current output row. For each level:

- Pop exactly the nodes that are currently in the queue.
- Add their children to the queue for the next level.
- Insert each popped value into the current row according to the direction.

Using a `deque` for the row lets us append to the right for left-to-right levels and append to the left for right-to-left levels, avoiding an extra reverse step per level.

6. Optimal Solutions Ranked

Rank	Variant	Idea	Time	Space	Tradeoff
1	BFS + reverse odd levels	Collect each level left-to-right, then reverse that list when the direction is right-to-left.	$O(n)$	$O(n)$	Simple and very readable; each node value may participate in a level reversal.

Rank	Variant	Idea	Time	Space	Tradeoff
2	BFS + row deque	Collect each level into a deque, appending right or left depending on direction.	$O(n)$	$O(n)$	Avoids explicit reversal and expresses the zigzag rule directly.
3	Two-stack traversal	Use one stack for the current level and one for the next level, pushing children in different orders.	$O(n)$	$O(n)$	Also optimal, but easier to get child order wrong and less directly tied to standard BFS.

All meaningful variants must touch every node at least once, so no algorithm can beat $O(n)$ time. The result itself may contain all n values, so $O(n)$ output space is unavoidable. The final implementation uses BFS with a per-level deque because it is optimal and clean.

7. Most Optimal Approach

Use a queue to perform level-order traversal. For every loop iteration, the queue contains exactly one level's nodes. Process those nodes, build a row in the correct direction, then enqueue their children for the next level.

The invariant is:

At the start of each outer loop iteration, the queue contains all nodes of the current depth in normal left-to-right order.

That invariant lets us separate traversal order from output order.

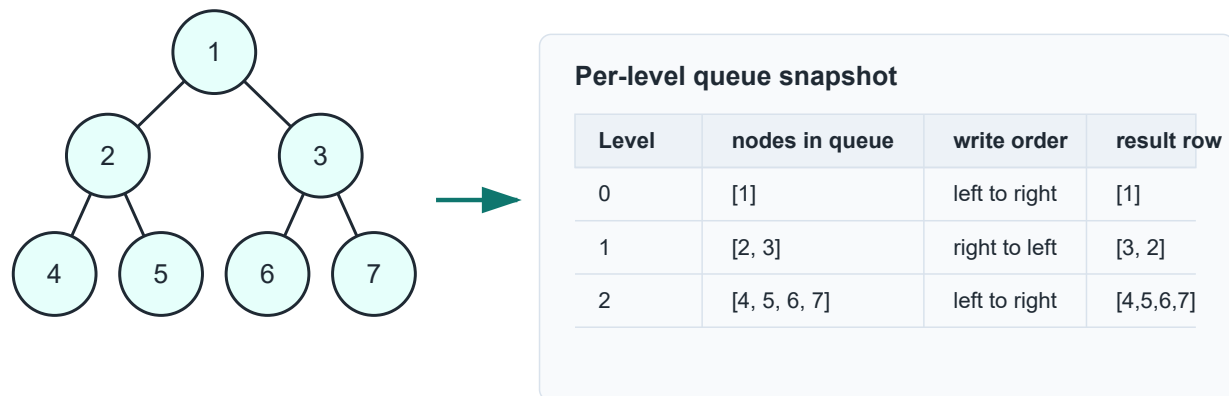
8. Algorithm

1. If `root` is `None`, return `[]`.
2. Initialize `queue = deque([root])`.
3. Initialize `left_to_right = True`.
4. While the queue is not empty:
 1. Store `level_size = len(queue)` so only the current level is processed.
 2. Create an empty `level = deque()`.

3. Repeat `level_size` times:
 1. Pop the next node from the queue.
 2. If `left_to_right` is true, append the value to the right of `level`.
 3. Otherwise, append the value to the left of `level`.
 4. Enqueue the node's left child if it exists.
 5. Enqueue the node's right child if it exists.
4. Convert `level` to a list and add it to the answer.
5. Flip `left_to_right`.
5. Return the answer.

9. Visual Explanation

Example: process levels normally, write values in alternating directions

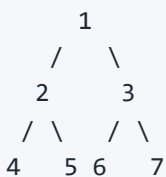


The child enqueue order stays left child then right child. Only the row insertion direction changes.

Figure 1: BFS still discovers children left to right; only the current result row changes insertion direction.

10. Dry Run

Use this tree:



Step	Queue at start	Direction	Row construction	Answer so far
1	[1]	left to right	append 1 -> [1]	[[1]]
2	[2, 3]	right to left	append-left 2, append-left 3 -> [3, 2]	[[1], [3, 2]]
3	[4, 5, 6, 7]	left to right	append right each value -> [4, 5, 6, 7]	[[1], [3, 2], [4, 5, 6, 7]]

Final output:

```
[[1], [3, 2], [4, 5, 6, 7]]
```

11. Correctness Argument

We prove the algorithm returns the zigzag level order traversal.

Invariant: At the start of each outer loop iteration, `queue` contains exactly the nodes of the current tree level in left-to-right order.

Initialization: Before the first iteration, the queue contains only `root`, which is exactly level 0 in left-to-right order.

Maintenance: During one iteration, the algorithm processes exactly `level_size` nodes, so it processes only the current level. For each node, it enqueues the left child before the right child. Since the current level is processed left to right, the next level is enqueued left to right. Therefore the invariant holds for the next iteration.

Output order: For each level, if the direction is left to right, values are appended to the end of the row, preserving queue order. If the direction is right to left, values are appended to the front of the row, reversing queue order. The direction flag flips after every level, so rows alternate exactly as required.

Termination: The loop ends after every reachable node has been removed from the queue once. By the invariant and output order reasoning, every level has been added to the answer in the required zigzag direction.

Therefore, the algorithm is correct.

12. Complexity Analysis

- Time: $O(n)$, where n is the number of nodes. Each node is popped from the queue once and inserted into one result row once.
- Space: $O(n)$. The returned answer stores all node values. The queue can also hold up to one tree level, which is $O(n)$ in the worst case.

13. Edge Cases

- Empty tree: `root = []` returns `[]`.
- Single node: `root = [1]` returns `[[1]]`.
- Skewed tree: each level has one node, so the output is still one value per row.
- Duplicate values: values are copied as they appear; duplicates do not affect traversal.
- Negative values: handled normally because no arithmetic comparison is needed.

14. Final Clean Code

```
from collections import deque
from typing import List, Optional

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if root is None:
            return []

        result = []
        queue = deque([root])
        left_to_right = True

        while queue:
            level = deque()

            for _ in range(len(queue)):
                node = queue.popleft()

                if left_to_right:
                    level.append(node.val)
                else:
                    level.appendleft(node.val)

                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)

            result.append(list(level))
            left_to_right = not left_to_right

        return result
```

15. Key Takeaways

- Main pattern: BFS level-order traversal.
- Main trick: Keep traversal order normal, but alternate how values are inserted into the current row.
- Interview explanation: "The queue gives me levels. A direction flag decides whether I append values normally or to the front of a row deque. Then I flip the flag after each level."